

FPGA experiment report

Task: FPGA

Student 1 name: Ariel Turnowski ID:206483513
Student 2: name: Ofek Goshen ID:313265647

Part 1 - Status update

Fill in the following table according to your progress at the time of the submission.
Each row should refer to one of the modules or a simulation. For example, write FA under module name and refer to FA.v and FA_tb.v in that line.

The status column should be filled with one of the following:

- Done - If both are written and simulation passed.
- Partially done - If you started writing the module and simulation but it's not working properly.
- Not started - If you didn't get to that part of the task.

When the status is partially done, please use the notes/comments column to explain in a few more words what is the status of the module and what is the status of the simulation.

Fill here:

Module name/Step	status	notes/comments
FA	DONE	
CSA	DONE	
Limited Incrementor	DONE	
Counter	DONE	
Stash	DONE	
Control	DONE	
Debouncer	DONE	
Seven Segment Display	DONE	

Stopwatch	DONE	
Setting up constraints	DONE	
Implementing the design	DONE	
FPGA programming and debugging	DONE	

Issues:

In this section please describe the current issues you are facing.

- Per each module and simulation, if you receive any errors at the time of submission, please describe which errors and what they mean.

Fill here:

1. FA:
2. CSA:
3. Limited Incrementor:
4. Counter:
5. Stash:
6. Control:
7. Debouncer:
8. Seven Segment Display:
9. Stopwatch:
10. Setting up constraints:
11. Implementing the design:
12. FPGA programming and debugging:

Part 2 - Simulations and answers

In this section, please add the images for each simulation as described in the report. Remember to use annotations and explain in your own words what is shown in each simulation.

In some of the tasks in the report, there are also questions that you should answer here.

Make sure to include images of all simulations running at the time of submission, even if they are not correct.

Fill here:

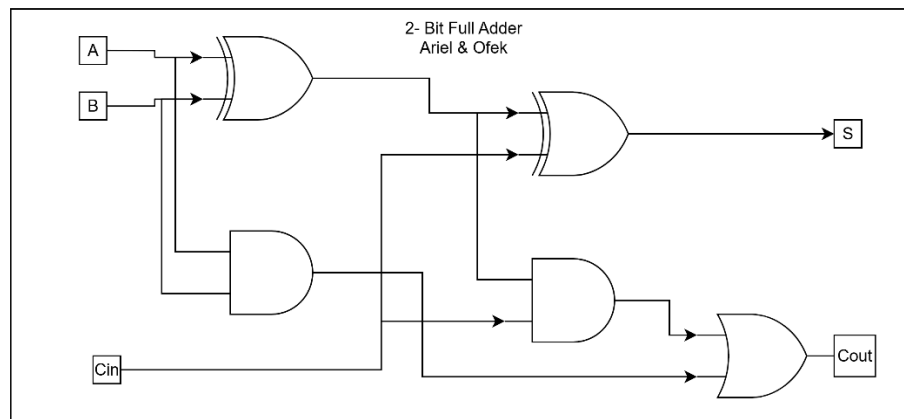
1. FA:

- .v files are provided.
- Full adder truth table:

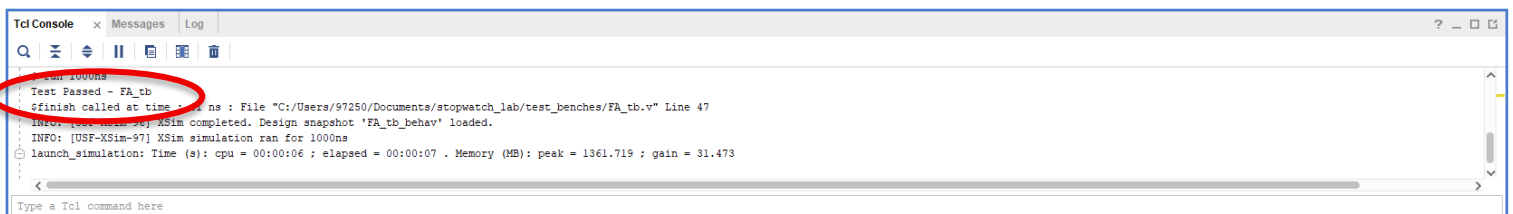
The rows the produce carry out are rows where $a + b + ci \geq 2$ (rows 4,6,7,8)

Input			Output	
a	b	ci	sum	co
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

c.

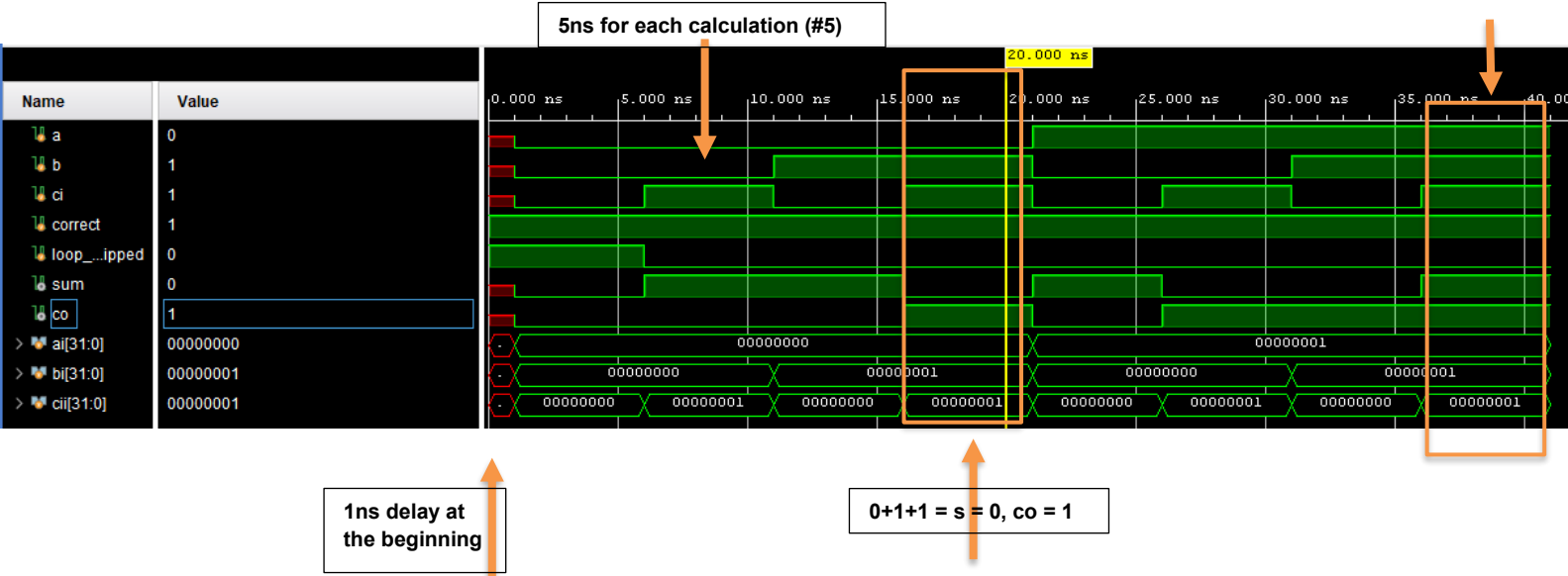


- Filled FA.v is provided in the design files.
- Test passed test bench:



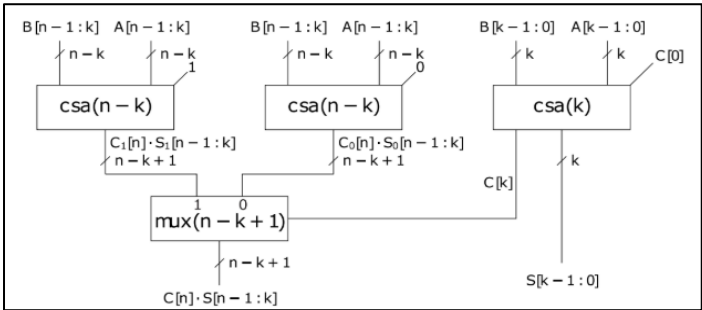
$1+1+1 = s=1, co = 1$

f. Noted waveform:

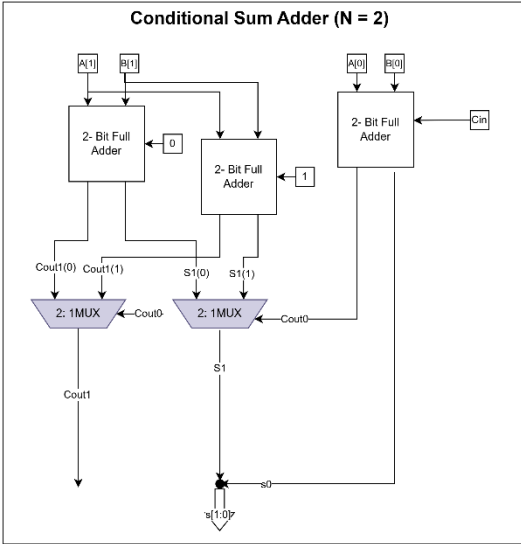


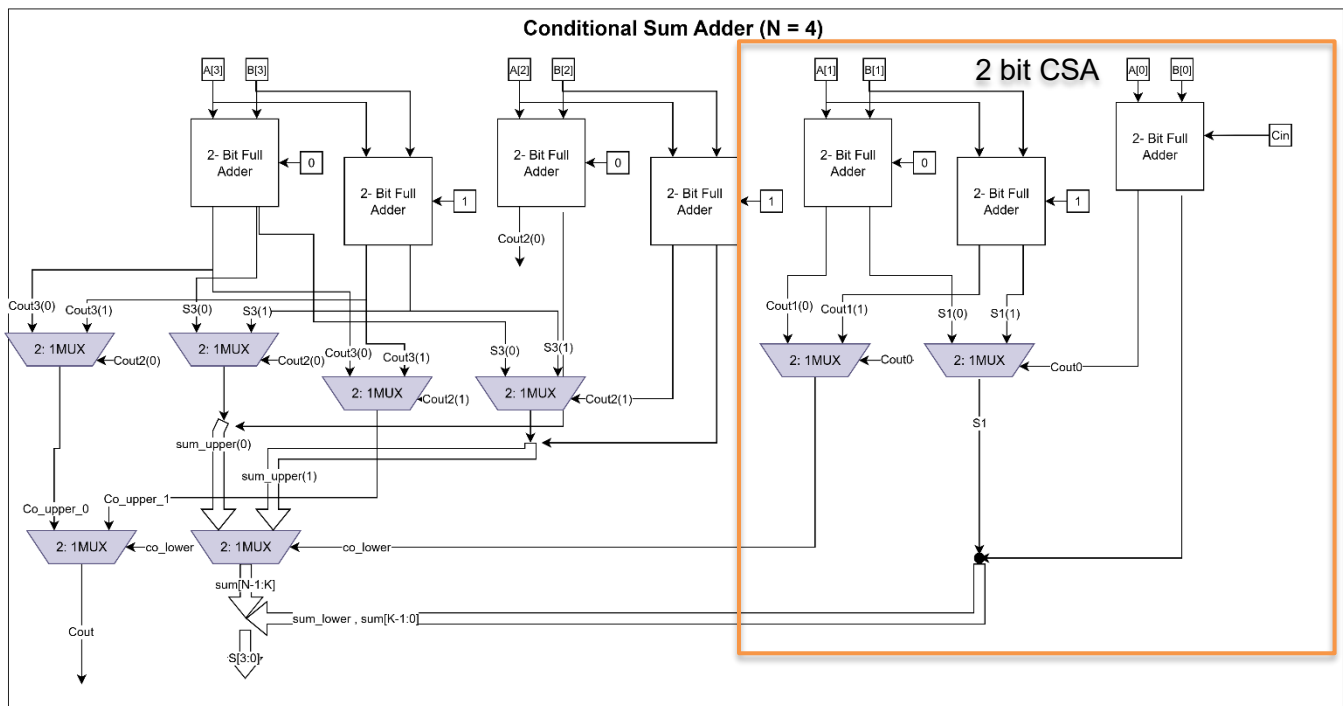
We can observe the simulation results, first we start with a 1ns delay at the beginning and the iteration for the truth table begins, for each iteration we have 5ns time as the tb instructs. loop_was_skipped is set to 0 is asserted to 0 when the first calculation starts, we can observe the correctness of the design with outputting correct co and sum bits.

2. CSA:
- a. For the general recursive case:



we calculate both carry cases for the second bit
When its bit is ready we choose the right one according to its carry (mux selector)





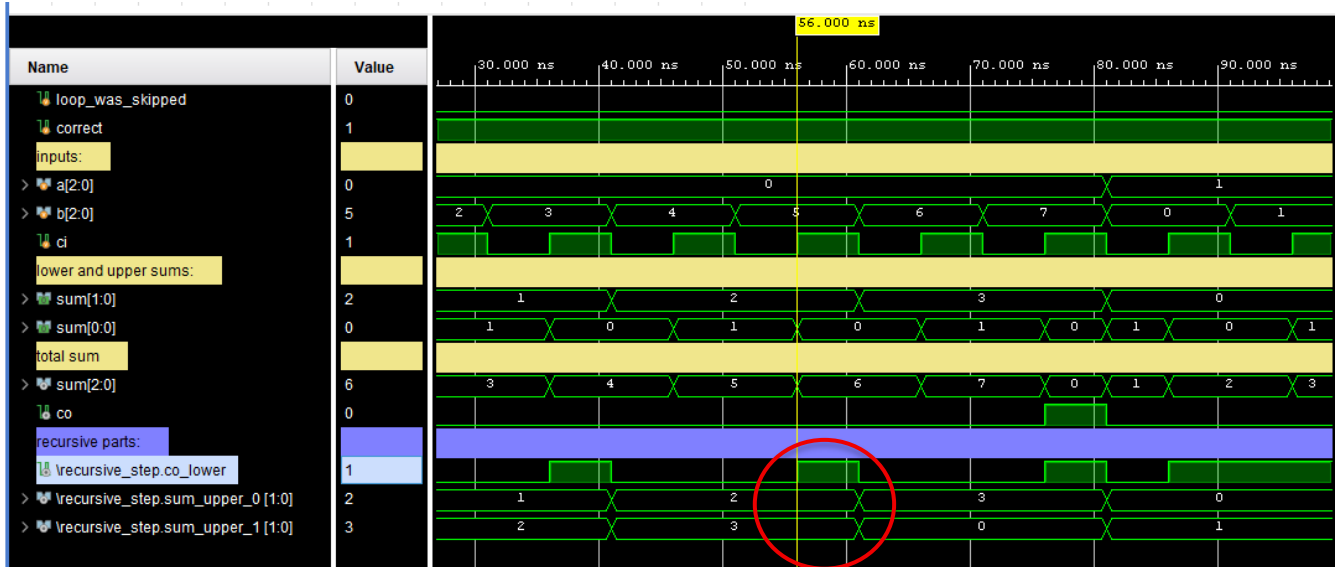
- b. CSA.v and CSA_tb.v files are provided in the design deliverables.
- c. CSA improve timing by dividing the adder into blocks. Each block precomputes results for both possible carry-in values (0 and 1) in parallel. Once the actual carry-in is known, a multiplexer selects the correct result, instead of waiting for the carry to propagate through every full adder (FA). A ripple-carry adder may be preferable for small bit-width designs where simplicity, low area and power consumption, non-critical timing paths, and limited hardware resources make high-speed operation unnecessary. CSA – faster timing, larger area RCA – longer latency and cheaper in area.
- d. Generate in Verilog is used to replicate hardware structure multiple times, just like in our case for the recursive CSA structure. In our case it is replicated 3 times.
- e. We designed the CSA according to the instructions.
- f. test bench results:

```

Tcl Console x Messages Log
[Icons]
# }
# }
# sum 1000ns
test Passed - CSA_tb
$finish called at time : 641 ns : File "C:/Users/97250/Documents/stopwatch_lab/test_benches/CSA_tb.v" Line 64
INFO: [USF-XSim-96] XSim completed. Design snapshot 'CSA_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:05 ; elapsed = 00:00:08 . Memory (MB): peak = 1359.594 ; gain = 37.332

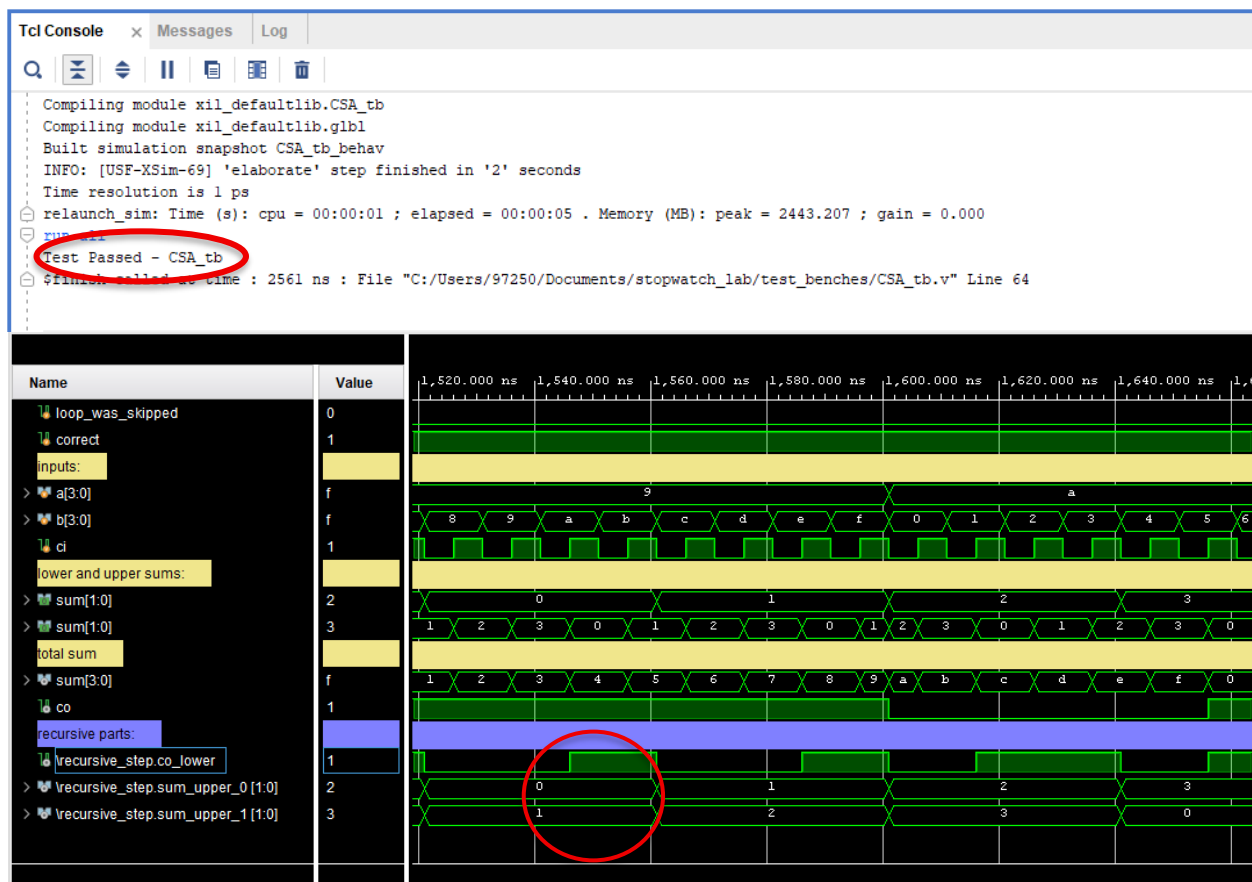
```

g. Waveform simulation:



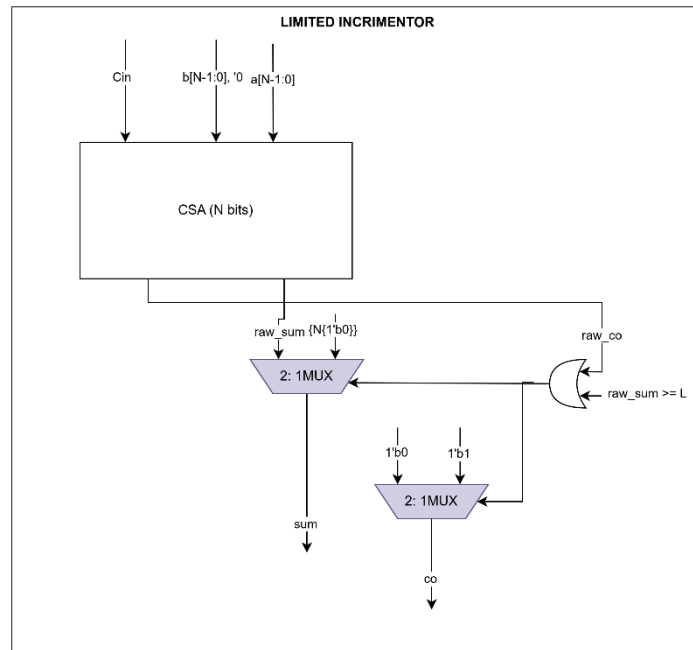
we can see that observe that when we get the `co_lower` to be active – we switch the muxes outputs it indeed changes the total sum by 1, since we add the one extra carry bit. We can also observe that we we are calcualte an output of 1000, the `co_lower` goes up and the carry output mux changes from 0 to 1.

h. For N=4, same behavior where `co_lower` changes the sum changes.



3. Limited Incrementor:

a.



b. We provided the Lim_Inc.v and tb in the files.

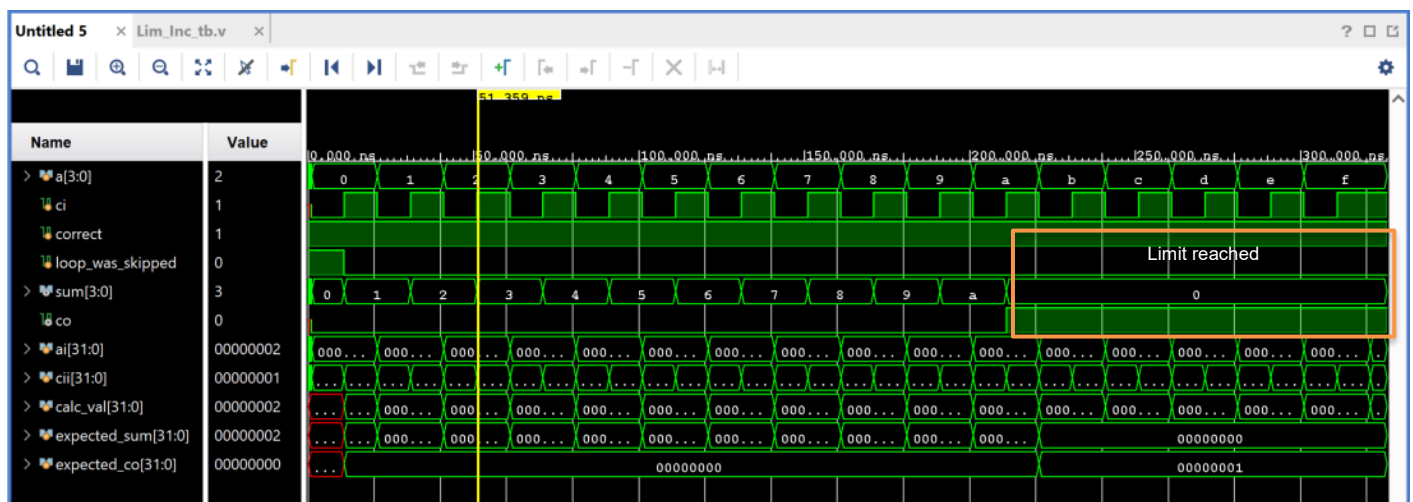
c. We created the module as required

d. Test and waveforms: we can see that we count 0 to 6 and then get 0 as intended.

```

Tcl Console  x Messages  Log
# run 1000ns
Test Passed - Lim_Inc_tb
$finish called at time: 326 ns : File "C:/Users/97250/Documents/stopwatch_lab/test_benches/Lim_Inc_tb.v" Line 98
INFO: [USF-XSim-96] XSim completed. Design snapshot 'Lim_Inc_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:05 ; elapsed = 00:00:16 . Memory (MB): peak = 4362.969 ; gain = 0.000

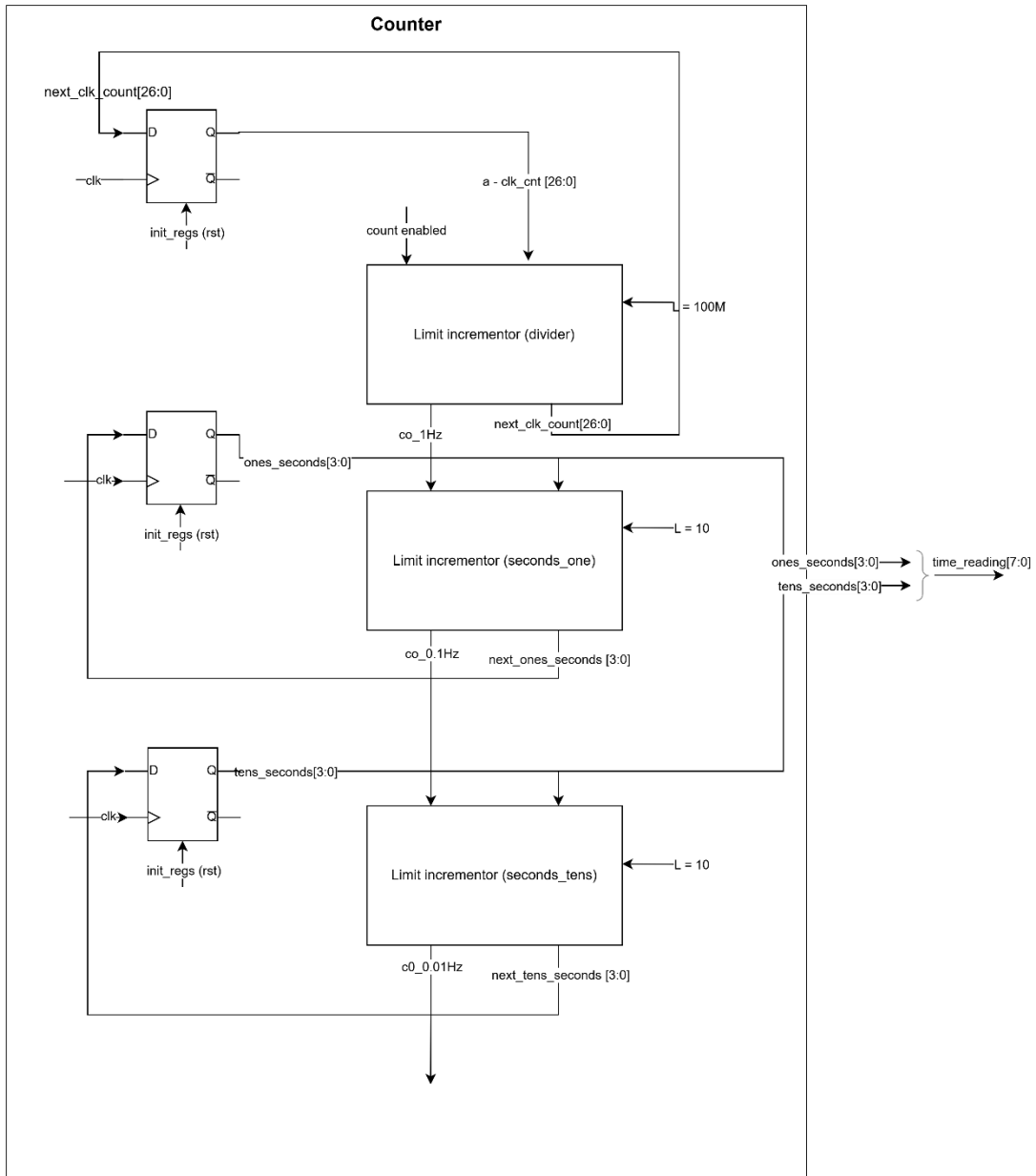
```



- e. We can see that raw_sum and the outputed sum are different (in orange limit reached box) that's since the raw sum is the arithmetic addition sum at the output of the CSA and in our module we want to count until the limit (in this case 11). There's a case where they never differ – when the limit is a power of 2 and that's actually a regular full adder logic with carry and sum, for example if $L = 4' d8 = 1000$ so sum and raw sum are 3 bits so we will reach the limit at the same cycle as the overflow will rise.

4. Counter:

- a. The time range that can be represented is 00 – 99 seconds, we separate the 8 bits to have 4 bits for the seconds and deci seconds so we can reach 0 to 9 for each one.
- b. In order to toggle the counting of the seconds when use a 100MHz clock, we will use a register of 27 ($\sim \log(100M)$) bits and the Lim_Inc module in the following way:
The register will store the current number of elapsed clock cycles. This register's output is connected to the input a of the Lim_Inc module. The Lim_Inc Module will be used with a limit parameter of $L = 100\text{Mega}$. The sum output of the Lim_Inc is looped back to the input of the register. On every clock edge (if counting is enabled), the register updates with the incremented value.
The Toggling Mechanism: The Lim_Inc continually increments the count as long as count_enabled is high. When the count reaches the limit L, the Lim_Inc asserts its carry-out (co) to 1 and resets the sum to 0. This co signal serves as the 1 Hz "tick" that toggles the next counter (the Seconds Counter) by driving its carry-in (ci) input.
- c. We will need 3 Lim_Inc modules in the design:
Lim_Inc n1 (Divider): Counts up to 100,000,000 , Lim_Inc n2 (Seconds): Counts 0-9 (triggered by the divider), Lim_Inc n3 (Tens of Seconds): Counts 0-9 (triggered when Seconds overflows).
- d. The parameter L should be set in our design to 10 since we want to generate a 0.1 Hz signal from 1 Hz signal .In general case if we want to generate the 0.1Hz from the first stage we need to set L to $f/0.1$. As mentioned, We do not generate the 0.1 Hz tick directly from the 100 MHz clock. Instead, you use the output of the previous stage (the Seconds Counter).
- e. Only the parameter value L for the first Lim_Inc module (the frequency divider) would change to 50,000,000. The design connections would remain the same.
- f. Block diagram : (next page)



- g. Added files in submission
- h. Added files in submission

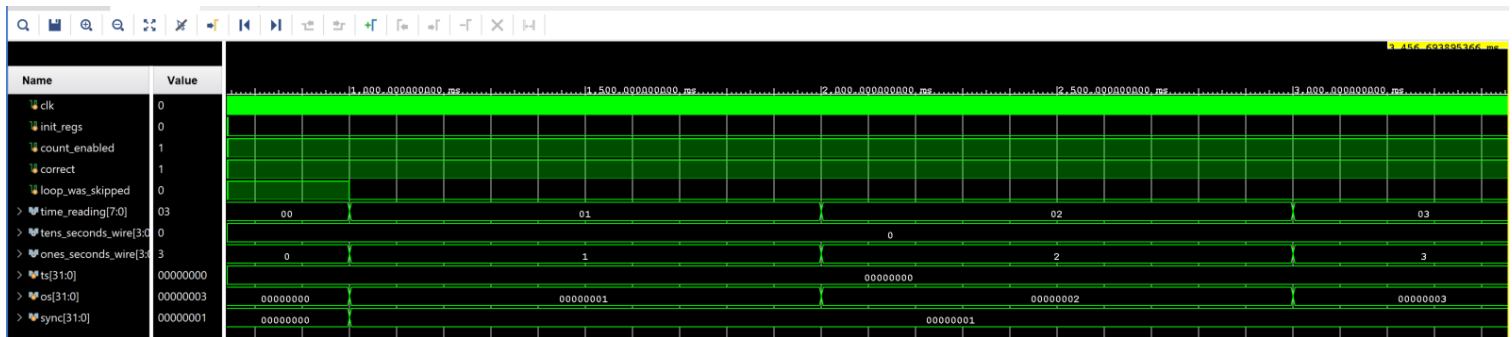
i. test log:

```

Messages Log Tcl Console x
# run 1000ns
INFO: [USF-XSim-96] XSim completed. Design snapshot 'Counter_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:02 ; elapsed = 00:00:05 . Memory (MB): peak = 2448.434 ; gain = 0.000
Test Passed - Counter_tb
$finish called at time : 200002237 ns : File "C:/Users/97250/Documents/stopwatch_lab/test_benches/Counter_tb.v" Line 91
run: Time (s): cpu = 00:06:52 ; elapsed = 00:09:21 . Memory (MB): peak = 2448.434 ; gain = 0.000

```

j. Waveforms: we can observe the time reading change,



- k. The Lim_Inc module cannot hold its own count because it is a purely combinational circuit. A counter requires a feedback loop where the output (Next State) affects the input (Current State). If you connected the output of the Lim_Inc directly back to its input without an intervening register, you would create an unstable combinational loop (race conditions), causing the value to race uncontrollably rather than counting once per clock cycle. To make the count increment exactly once per clock cycle (or event), an external register is required to hold the value stable between clock edges.
- l. The maximum time the counter can represent is 99 seconds.
- m. In order to implement (MM:SS) counter – we will need to add two more layers, The Carry-Out (co) of each stage would drive the Carry-In (ci) of the next stage: Divider triggers Sec Units (L= 10), Sec Units triggers Sec Tens (L=6), Sec Tens triggers Min Units (L=10), Min Units triggers Min Tens (L=10).

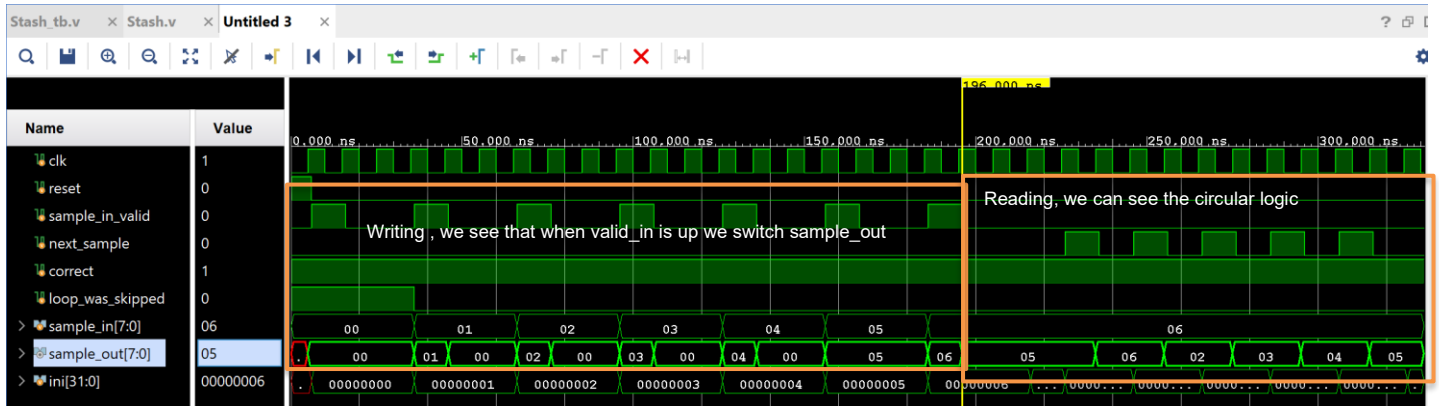
Stash

The Stash module diagram shows a memory array with 5 rows and 4 columns, indexed by `wr_ptr` and `rd_ptr`. The memory contents are 100, 011, 010, 001, and 000. A 2-to-1 MUX selects between `memory[wr_ptr]` and `memory[rd_ptr]` based on the `showing_latest` signal (timed to half a second 50M clocks). The output is `sample_out`. A second 2-to-1 MUX selects between `write_ptr` and `next_write_ptr_val` based on `sample_in_valid`. The output of this MUX is the `D` input of a D flip-flop. The flip-flop's `Q` output is `write_ptr`. A third 2-to-1 MUX selects between `read_ptr` and `next_read_ptr_val` based on `next_sample`. The output of this MUX is the `D` input of another D flip-flop. The flip-flop's `Q` output is `read_ptr`. Both flip-flops are reset by a `reset` signal. The `Limit incrementor (write logic)` block takes `write_ptr` and `sample_in_valid` to produce `next_write_ptr_val`. The `Limit incrementor (read logic)` block takes `read_ptr` and `next_sample` to produce `next_read_ptr_val`.

- b. Weve added the .v files
- c. We designed the stash by using the Lim_inc module it is used to maintain the pointer values between 0 and 4 (size 5) and incremented when we push the button.
- d. Test log:

```
Tcl Console x Messages Log
# }
run 1000ns
Test Passed - Stash_tb
finish called at time : 81 ns : File "C:/Users/97250/Documents/stopwatch_lab/test_benches/Stash_tb.v" Line 84
INFO: [USF-XSim-36] XSim completed. Design snapshot 'Stash_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:06 ; elapsed = 00:00:09 . Memory (MB): peak = 1512.109 ; gain = 9.398
```

- e. Waveforms: (in the tb we show the wr pointer cell value for a cycle and not half a second since we want a quick test and not a long heavy test)



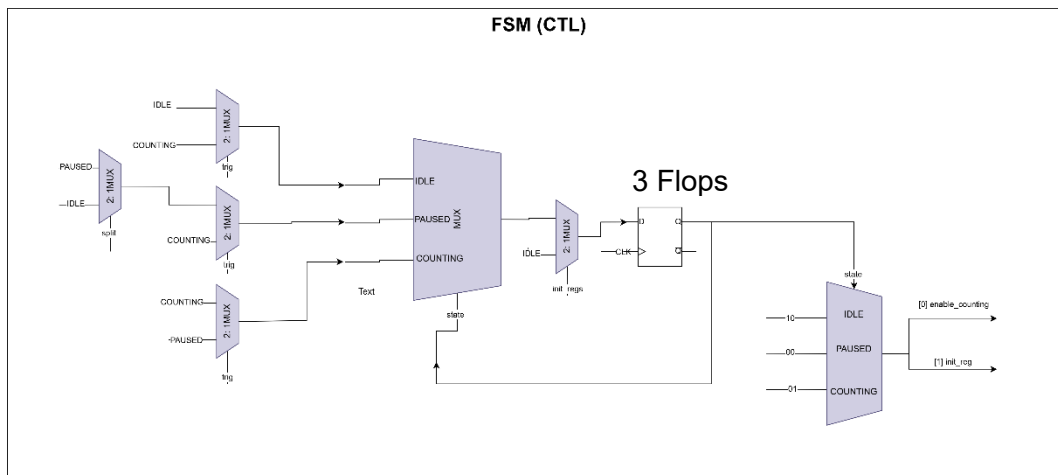
- f. Sample_out is assigned by a **mux** (combinational):
`assign sample_out = (showing_latest) ? captured_sample : memory[read_ptr];`
 which instantly sets its value as needed.
 showing latest – triggers on button click, counts half a second to show the written data and then muxes back to the read ptr
 captured sample – the sample captured in the live time when the button was clicked.
- g. We can use ptr muxing (address muxing) instead of the data it self.
`addr = (showing_latest) ? write_ptr -1 : read_ptr;`

	Data muxing	Address Muxing
Pros	Path is Faster and simpler in RTL : Input Pad -> Mux -> Output.	Smaller mux simpler in physical implementation
Cons	Physical connections are more complicated. need bigger mux	Path is slower because of memory read: Ptr Logic -> Memory Read Access -> Output. We need to add another register to remember the address we want to get back to.

6. Control:

- The FSM is represented with a Mealy state diagram, it can be also represented with a Moore state diagram – that's because we can see that the outputs do not dependent on the inputs at all, but only on the **state**. For every state we get a certain output no matter the input (for IDLE the output is '10', for COUNTING the output is '01' and for PAUSED the output is '00').

b.

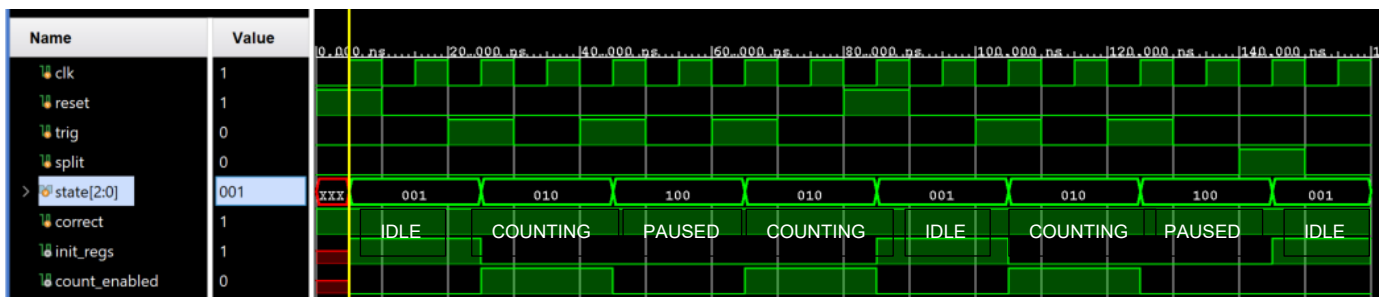


- Weve added the .v files in the files
- Weve added the .v files in the files
- Test log

```
Tcl Console x Messages Log
# }
# }
# run 1000ns
Test Passed - Ctl_tb
$finish called at time : 160 ns : File "C:/Users/97250/Documents/stopwatch_lab/test_benches/Ctl_tb.v" Line 124
INFO: [USF-XSim-96] XSim completed. Design snapshot 'Ctl_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
```

f. Waveform

We can observe that when reset is held, the state stays at IDLE trig is asserted and we move to COUNTING, trig is asserted again and we PAUSE, then we trig again and we move to COUNTING again. We then reset the clock and get the init_regs up and go to IDLE , we start again the COUNTING .



8. Seven Segment Display:

- a. The provided design source Seg_7_Display.v is added to our project.
- b. We edited the code to be: (line 69) assign s = clkdiv[19:18];
- c. We know that only one digit of the 4 can be turned on and off at a given time, and so we require total refresh time of all 4 digits must be between 16ms and 1ms for a human eye to see the display properly (with all 4 digits). We need to find which bits of the counter to use to control this switching speed. **Our Clock:** 100 MHz means 1 tick = **10 nanoseconds (ns)**. **We'll** aim for **10.5 ms** for a full cycle (a safe middle value).

We need to check how many 10ns ticks fit into 10.5ms:

$$\begin{aligned} 10.5 \text{ ms} &= 10,500,000 \text{ ns} \\ \frac{10500000 \text{ ns}}{10 \frac{\text{ns}}{\text{tick}}} &= 1050000 \text{ ticks} \end{aligned}$$

we look for a binary bit 2^n that is close to 1,050,000:

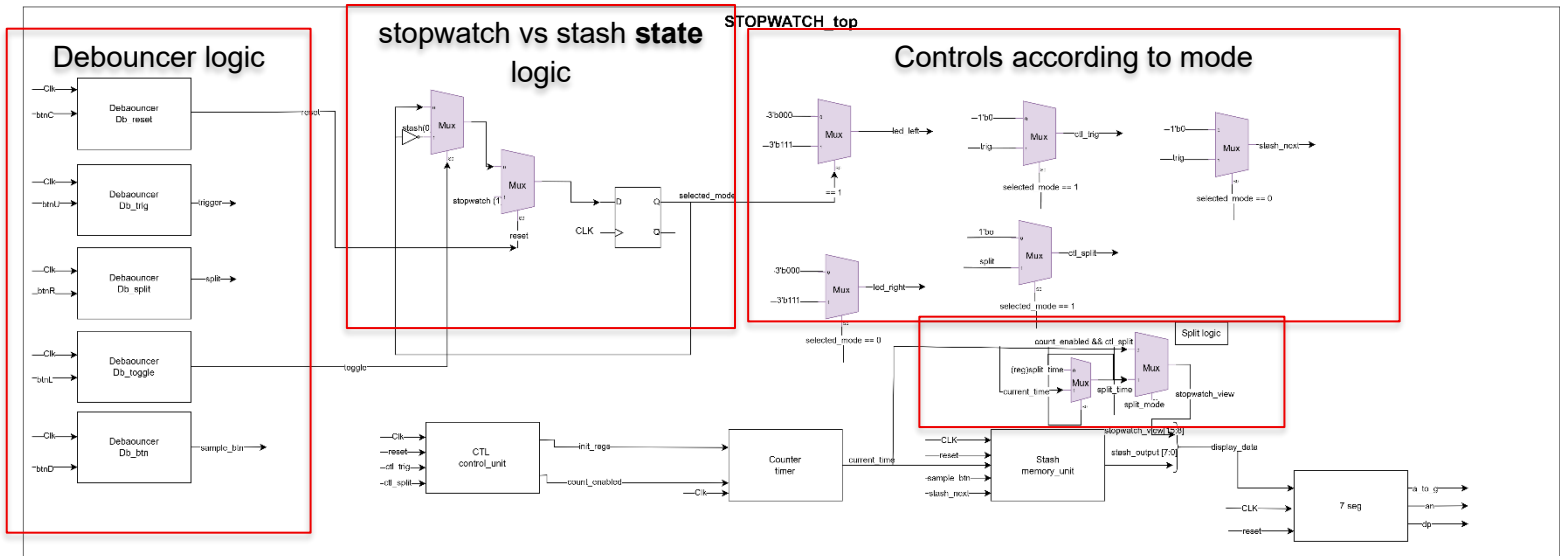
$$2^{18} = 262144, 2^{19} = 524,288$$

So, if we use the **18th and 19th bits** of the counter as our "Selectors" - The counter counts up and bits 19-18 change pattern roughly every 2.6 ms according to the lsb.

We get *Total cycle time* = $2.6 \text{ ms} \cdot 4 = 10.4 \text{ ms}$, which satisfies our goal.

9. Stopwatch:

a.



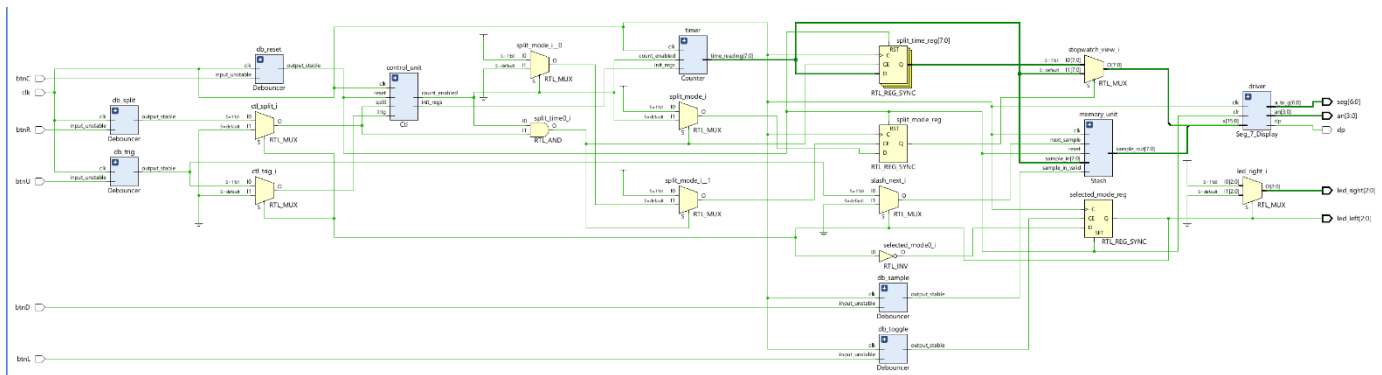
from the left we see the Debouncer logic one instance for every button, then the mode state logic, chooses if we are in stash or stopwatch modes according to the reset or toggle buttons, then we have the 5 muxes that determine the leds, stash, split and trigger according to the mode. At the bottom we see the main instances of the device. The control, the counter, the stash, the split logic and the 7 segment display before the output

b. We've added the .v files to our files.

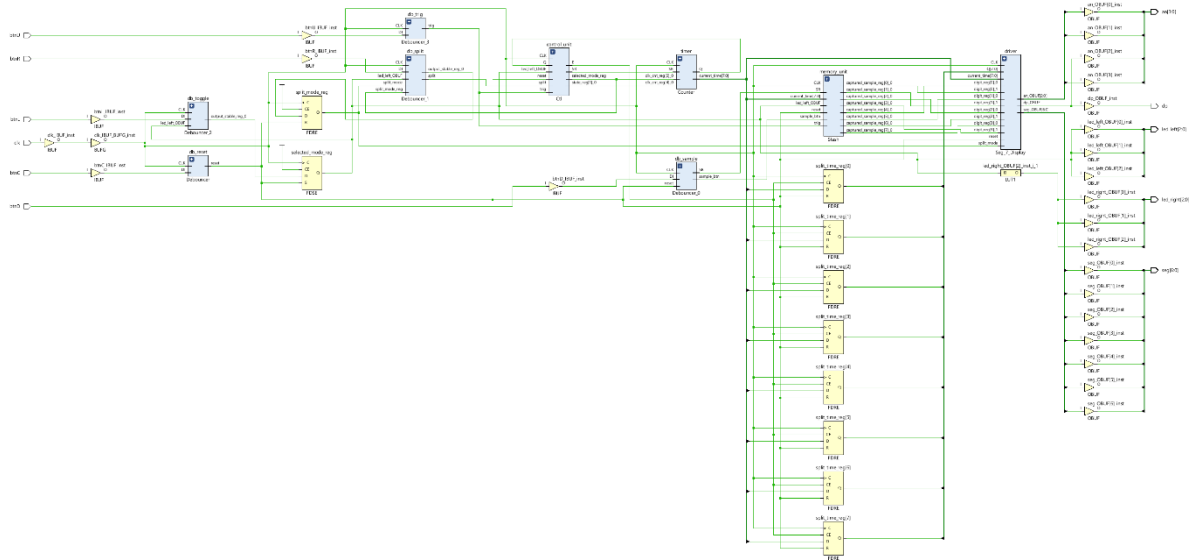
c. .

d. .

e. The elaborated schematic (top level):



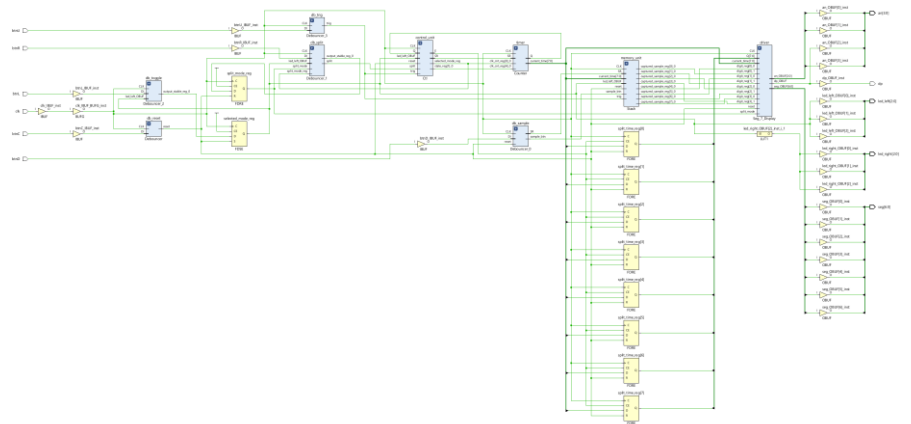
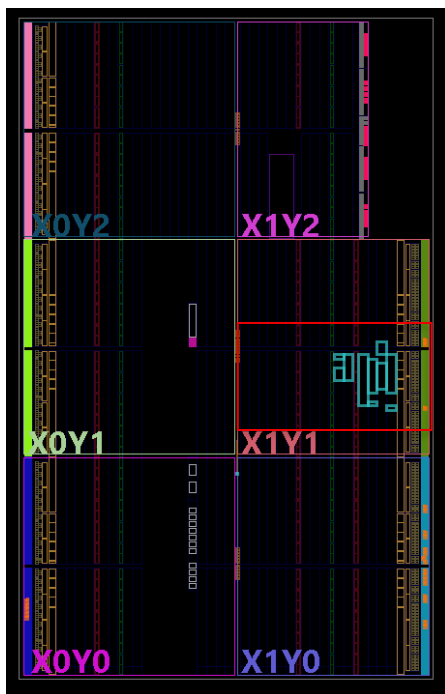
- f. In the synthesis schematic we can see that buffers were added, to align timing we can also see that generic RTL blocks are replaced by primitive cells available on the FPGA such as LUT (Look-Up Tables), FDRE (Flip-Flops), and CARRY4 chains.



10. Setting up constraints

11. Implementing the design:

c. schematics:



d. Timing summaries:

Tcl Console

Messages

Log

Reports

Design Runs

Methodology

Power

Linter

Timing x

Design Timing Summary

General Information

Timer Settings

Design Timing Summary

Clock Summary (1)

Methodology Summary (23)

Check Timing (23)

Intra-Clock Paths

Inter-Clock Paths

Setup

Hold

Pulse Width

Worst Negative Slack (WNS): 2.528 ns

Total Negative Slack (TNS): 0.000 ns

Number of Failing Endpoints: 0

Total Number of Endpoints: 467

Worst Hold Slack (WHS): 0.178 ns

Total Hold Slack (THS): 0.000 ns

Number of Failing Endpoints: 0

Total Number of Endpoints: 467

Worst Pulse Width Slack (WPWS): 4.500 ns

Total Pulse Width Negative Slack (TPWS): 0.000 ns

Number of Failing Endpoints: 0

Total Number of Endpoints: 199

All user specified timing constraints are met.

Intra-Clock Paths - sys_clk_pin

Clock: sys_clk_pin

Statistics

Type	Worst Slack	Total Violation	Failing Endpoints	Total Endpoints
Setup	2.528 ns	0.000 ns	0	447
Hold	0.178 ns	0.000 ns	0	447
Pulse Width	4.500 ns	0.000 ns	0	199

We can observe that our worst signal arrives 2.528 ns earlier (setup) than the FPGA limit, and our worst signal changes 0.178ns after the clock edge, we can also observe that no violations are present.

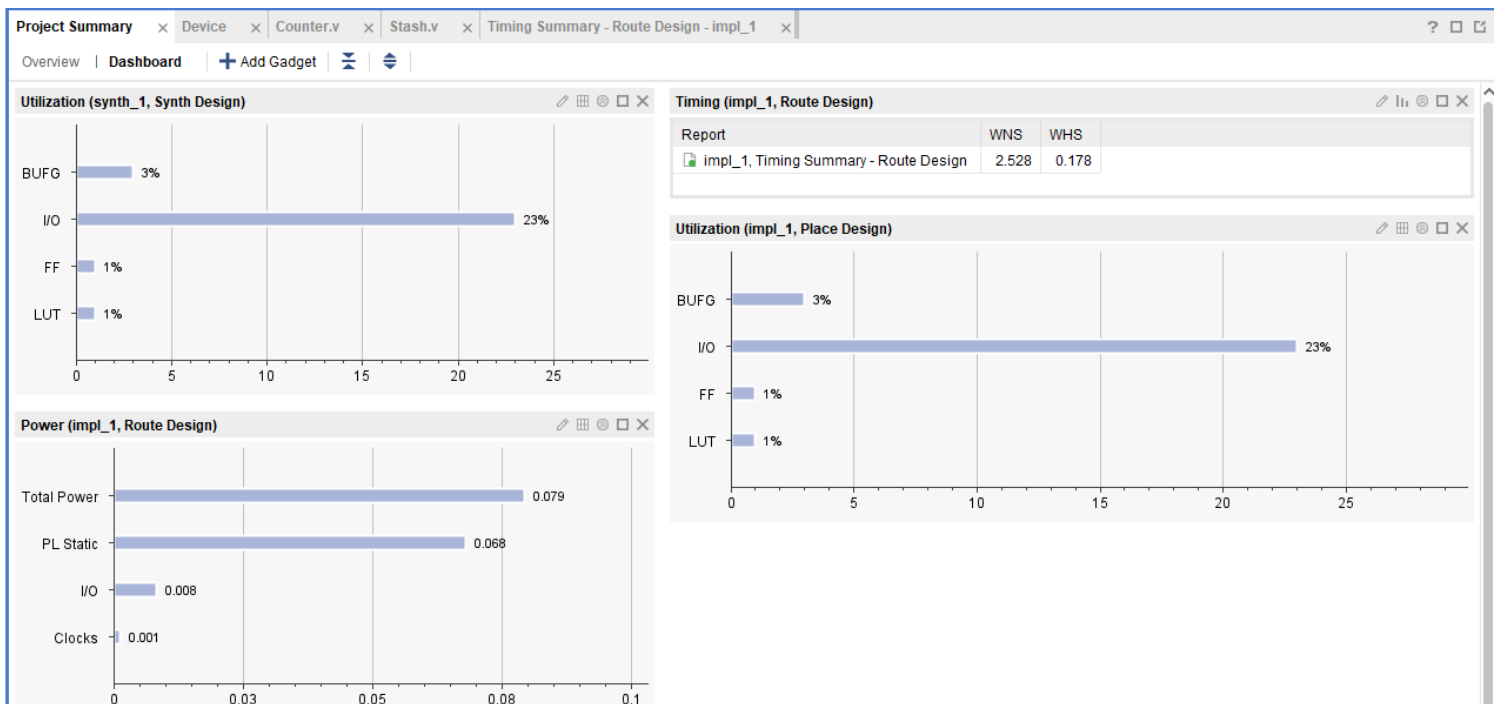
e. for the critical paths timings:

Tcl Console Messages Log Reports Design Runs Methodology Power Linter Timing x													
Other Path Groups - **async_default** - sys_clk_pin - Setup													
Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 21	8.031	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[10]/CLR	1.496	0.536	0.960	10.0	sys_clk_pin	sys_clk_pin		0.035
Path 22	8.031	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[11]/CLR	1.496	0.536	0.960	10.0	sys_clk_pin	sys_clk_pin		0.035
Path 23	8.031	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[8]/CLR	1.496	0.536	0.960	10.0	sys_clk_pin	sys_clk_pin		0.035
Path 24	8.031	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[9]/CLR	1.496	0.536	0.960	10.0	sys_clk_pin	sys_clk_pin		0.035
Path 25	8.066	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[16]/CLR	1.460	0.536	0.924	10.0	sys_clk_pin	sys_clk_pin		0.035
Path 26	8.066	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[17]/CLR	1.460	0.536	0.924	10.0	sys_clk_pin	sys_clk_pin		0.035
Path 27	8.066	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[18]/CLR	1.460	0.536	0.924	10.0	sys_clk_pin	sys_clk_pin		0.035
Path 28	8.066	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[19]/CLR	1.460	0.536	0.924	10.0	sys_clk_pin	sys_clk_pin		0.035
Path 29	8.206	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[12]/CLR	1.320	0.536	0.784	10.0	sys_clk_pin	sys_clk_pin		0.035
Path 30	8.206	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[13]/CLR	1.320	0.536	0.784	10.0	sys_clk_pin	sys_clk_pin		0.035
Other Path Groups - **async_default** - sys_clk_pin - Hold													
Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 31	0.475	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[0]/CLR	0.420	0.193	0.227	0.0	sys_clk_pin	sys_clk_pin		0.000
Path 32	0.475	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[1]/CLR	0.420	0.193	0.227	0.0	sys_clk_pin	sys_clk_pin		0.000
Path 33	0.475	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[2]/CLR	0.420	0.193	0.227	0.0	sys_clk_pin	sys_clk_pin		0.000
Path 34	0.475	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[3]/CLR	0.420	0.193	0.227	0.0	sys_clk_pin	sys_clk_pin		0.000
Path 35	0.625	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[4]/CLR	0.569	0.193	0.376	0.0	sys_clk_pin	sys_clk_pin		0.000
Path 36	0.625	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[5]/CLR	0.569	0.193	0.376	0.0	sys_clk_pin	sys_clk_pin		0.000
Path 37	0.625	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[6]/CLR	0.569	0.193	0.376	0.0	sys_clk_pin	sys_clk_pin		0.000
Path 38	0.625	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[7]/CLR	0.569	0.193	0.376	0.0	sys_clk_pin	sys_clk_pin		0.000
Path 39	0.653	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[12]/CLR	0.595	0.193	0.402	0.0	sys_clk_pin	sys_clk_pin		0.000
Path 40	0.653	0	101	db_reset/output_stable_reg/C	driver/clkdiv_reg[13]/CLR	0.595	0.193	0.402	0.0	sys_clk_pin	sys_clk_pin		0.000

From the data we got that our longest path is 8.031ns and the shorts path is 0.475 ns. We can note that the $longest_{path} + WNS < 10ns$ and that $Shortest_{path} \neq WHS$. The vivado automatically reduce the $T_{setup} \setminus T_{hold} + skew + jitter + clock_dely$ and the measurements we got are the actual different form the hardware limits.

12. FPGA programming and debugging:

Total dashboard summery, including power, area and timings.



working picture of the device:

